NAVIGATING SQL SERVER TO POSTGRESQL MIGRATION

And How DBAs Keep Their Sanity

redgate | About me

Kellyn Gorman

Multiplatform and AI Advocate

https://dbakevlar.com



THE REALITY

NUMBERS

Modernize…

- Marketing vs Reality
- Database vs. Ecosystem
- Workload vs. Vague Scenarios
- Vanilla vs. Native
- Whole vs. Slice n' Dice

What are the numbers I've experienced?

THE WHY

WHY MIGRATE?

- SQL Server Enterprise licensing pressure
  - Cloud and AI modernization initiatives
  - Open-source ecosystem growth
  - AI project alignment
  - Someone thought it just sounded cool!

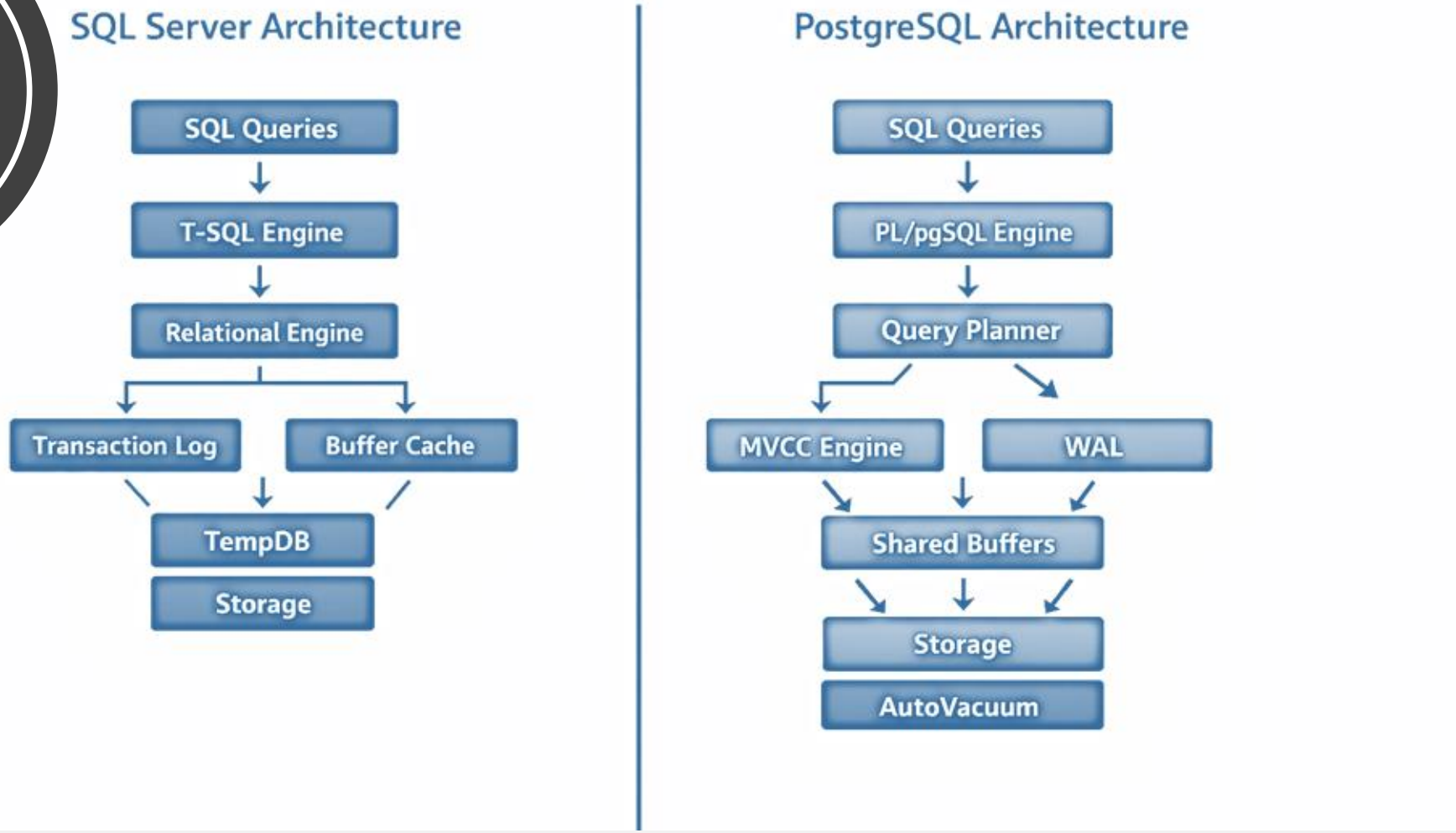


IT'S AN OPERATING MODEL SHIFT

Not just a platform change

- DBA mindset, tooling, and governance must evolve
- Subtle assumptions can break in painful ways
- Migration is architectural, not syntactic

HIGH LEVEL


SQL Server Architecture
SQL Queries
T-SQL Engine
Relational Engine
Transaction Log
Buffer Cache
TempDB
Storage
PostgreSQL Architecture
SQL Queries
PL/pgSQL Engine
Query Planner
MVCC Engine
WAL
Shared Buffers
Storage
AutoVacuum

THE WHAT

CONCURRENCY MODEL DEEP DIVE

- SQL Server: Lock-based, TempDB heavy
  - PostgreSQL: MVCC, tuple versioning
  - Long transactions cause bloat in Postgres
  - It's not better, but just different…

VACUUM is mandatory for health

PROCEDURAL CODE & ECOSYSTEM GAPS

- SQL Agent → pg_cron or external schedulers
- SSIS →Airflow / ELT/ETL
- CLR → rewrite required
- Linked Servers → postgres_fdw

TYPES OF POSTGRES

- Native
- Compatible
- Cloud
- Serverless

POSTGRES VARIATIONS & CLOUD LOCK-IN

- Community vs RDS vs Aurora vs Azure vs AlloyDB
  - Extension differences/limitations
  - Superuser restrictions
  - Storage & HA differences

- Choose flavor depending on requirements
- Test on the actual platform
- Validate extensions needed are available
- Validate HA architecture
- Verify lock-in implications

STORAGE & INDEX ARCHITECTURE

SQL Server: Clustered index defines table

- Postgres: Heap + secondary indexes
- BRIN, GIN, GiST, SP-GIST, HASH index types
- Clustered strategy must be rethought

THERE'S AN EXTENSION FOR THAT



SQL Server has CLR integration and .NET assemblies inside the database, but…



Postgres has extensions

PostGIS
pgvector
pg_stat_activity



PL/Python, PL/Perl



Existing CLR-heavy workloads do require redesign, not just translation.

MIGRATION IS NOT SYNTAX TRANSLATION

- T-SQL ≠ PL/pgSQL
  - Optimizer differences
  - Concurrency model differences
  - Statistics & memory management differences
  - Process and data load changes

STATISTICS & PLAN STABILITY

- SQL Server: Query Store, plan forcing
  - Postgres: Statistics target tuning
  - Extended statistics optional
  - Plan governance must be rebuilt

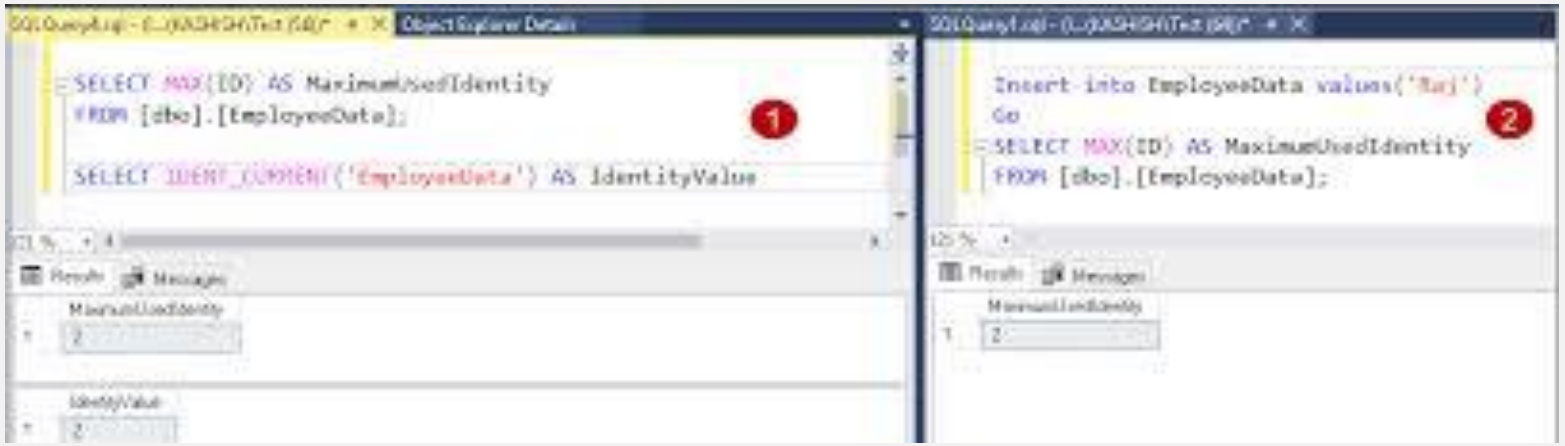
EXECUTION PLAN DIFFERENCES

- SQL Server: Memory grants, parameter sniffing
  - Postgres: work_mem sensitive, nested loop bias
  - Row estimation critical to performance
  - Plan stability requires new discipline

SCHEMA DIFFERENCES WITH DATA TYPES

UNIQUEIDENTIFIER → UUID

- DATETIME vs TIMESTAMP (timezone behavior)
- BIT vs BOOLEAN
- NVARCHAR vs TEXT/VARCHAR handling
- Implicit conversion behavior differences



IDENTITY COLUMNS AND SEQUENCES

IDENTITY vs GENERATED AS IDENTITY

- Sequence caching differences
- Gap behavior expectations
- Replication and failover considerations

ESCAPING LOCK-IN… INTO LOCK-IN

- Cloud-specific Architecture Pros/Cons
  - Managed service IOPS tiers
  - Superuser/OS-level lock-out
  - Automation of maintenance/backups
  - Different cloud versions often serve different purposes
  - Monitoring integration dependencies/restrictions
  - AI add-on billing layers

COST REDISTRIBUTION MODEL

- License removal ≠ total cost reduction
  - Cloud compute & IOPS
  - Scaling requirements after optimization loss
  - Read replicas & HA in multi-AZ environments
  - Ingress/egress charges/network enhancements
  - AI services & data pipelines
  - Application and code rewrites/refactoring
  - New Tooling licenses
  - Training

YOU DON'T REMOVE COST. YOU RELOCATE COST.

THE HOW

COMMON MIGRATION FAILURE POINTS

Underestimating query planner differences

- Ignoring autovacuum tuning
- Data type mismatches causing subtle bugs
- Lack of workload replay testing
- Declaring success without performance validation
- Defined cutover dates without understanding the nuances

VALIDATION AND PERFORMANCE

- Have a method to compare plan performance
  - Workload replay testing
  - Index usage analysis
  - Monitoring bloat and vacuum activity
  - Failover and HA testing WELL BEFORE cutover

PERFORMANCE VALIDATION FRAMEWORK

Inventory and classify workloads

- Baseline performance before migrations
- POC with production-like workload
- Consider parallel run validation

Replay real workloads

- Validate concurrency & failover
- Monitor vacuum & bloat
- Track plan regressions

Consider controlled and reversible cutover

AI USE CASE: STATIC CODE RISK SCORING

- Detect CLR & cross-database references
  - Flag transaction scope risks
  - Identify temp table heavy workloads
  - Generate object-level risk scores

AI USE CASE: QUERY PATTERN ANALYSIS

- Detect skewed predicates
  - Identify nested loop explosion risk
  - Highlight update churn tables
  - Predict MVCC bloat pressure

AI USE CASE: PLAN COMPARISON

- Compare SQL Server vs Postgres plans
  - Detect cardinality estimation drift
  - Highlight index differences
  - Flag regression risk before production

AUTOVACUUM & BLOAT RISKS

- Understand autovacuum thresholds
  - Monitor freeze age
  - Detect index bloat early
  - Cloud storage costs grow with bloat



MIGRATION TOOLS

HAVE TOOLS TO SUPPORT BOTH PLATFORMS

- Monitoring Visibility
- Migration for pipelines
- Automation for processes, transformations
- Clear communication with stakeholders
- Metric-driven validation

AI-ASSISTED MIGRATION RISK ANALYSIS

- Use AI to detect risk before migration
  - Analyze T-SQL complexity
  - Score rewrite effort
  - Flag architectural dependencies

RECOMMENDED MIGRATION FRAMEWORK

- Inventory & risk classification
  - AI-assisted analysis
  - Real-world POC testing
  - Parallel run & performance hardening

BEST DATABASE MIGRATION ASSISTANTS

- Pgloader with additional scripts
- AWS Transform for SQL Server Modernization
- Babelfish for Aurora PostgreSQL
- Ora2pg for SQL Server (with caveats)
- Google Database Migration Service (DMS)
- HexaRocket Migration Utility
- DBBridge with proprietary scripting

DEMO SCOPE



DBMIGRATE: ONE SCRIPT, END TO END

dbmigrate.py (DataBridge Migration Assistant v4.2.1) reads the SQL Server catalog, converts schema and T-SQL code, moves the data, and reports readiness for project RetailOps.

SOURCE

SQL Server 2022 (16.x)

SQLPROD01 · DB RetailOps
Enterprise, compat 160
Windows Integrated (Kerberos)



TARGET

PostgreSQL 17.9

pg-retailops-01 · DB retailops
schema public, UTF8 en_US
scram-sha-256 (user dbmigrate)

```
python dbmigrate.py
```
full run, presentation pacing

```
--fast
```
no pacing delays

```
--scale 2.0
```
2× row volume

```
--out ./out
```
folder for generated scripts/data

SEVEN PHASES FROM DISCOVERY TO READINESS

1 **Connect & Discover**

Connects to both ends, checks CREATE/TEMP/CONNECT, reads sys.tables, columns, indexes, FKs, sql_modules

2 **Assess**

Maps data types, scores 6 code objects, profiles 53,034 rows. Result: 0 blockers, 10 warnings, readiness 90/100

3 **Schema Conversion**

Tables to snake_case in public; identity → GENERATED BY DEFAULT; PKs, unique keys, FKs carried over

4 **Code Conversion**

T-SQL procs, function and view rewritten as PL/pgSQL, with the rules applied listed per object

5 **Data Migration**

Offline bulk COPY, 5,000-row batches, 4 parallel readers, triggers/FK checks suspended for load

6 **Post-load Validation**

Verifies the loaded target

7 **Readiness Summary**

Final report: migration_report.json

Type mapping from the Assess phase

BIT → BOOLEAN	UNIQUEIDENTIFIER → UUID
MONEY → NUMERIC(19,4)	NVARCHAR(MAX) → TEXT
DATETIME → TIMESTAMP(3)	DATETIME2(7) → TIMESTAMP(6)

WHAT THE RUN WRITES TO MIGRATION_OUTPUT

03_functions_views.sql

PL/pgSQL functions and views converted from T-SQL, each tagged with its source object

02_load_data.sql

\copy commands that load each table from data/<table>.csv

data/*.csv

One CSV per table, with cleansing rules applied during extract

migration_report.json

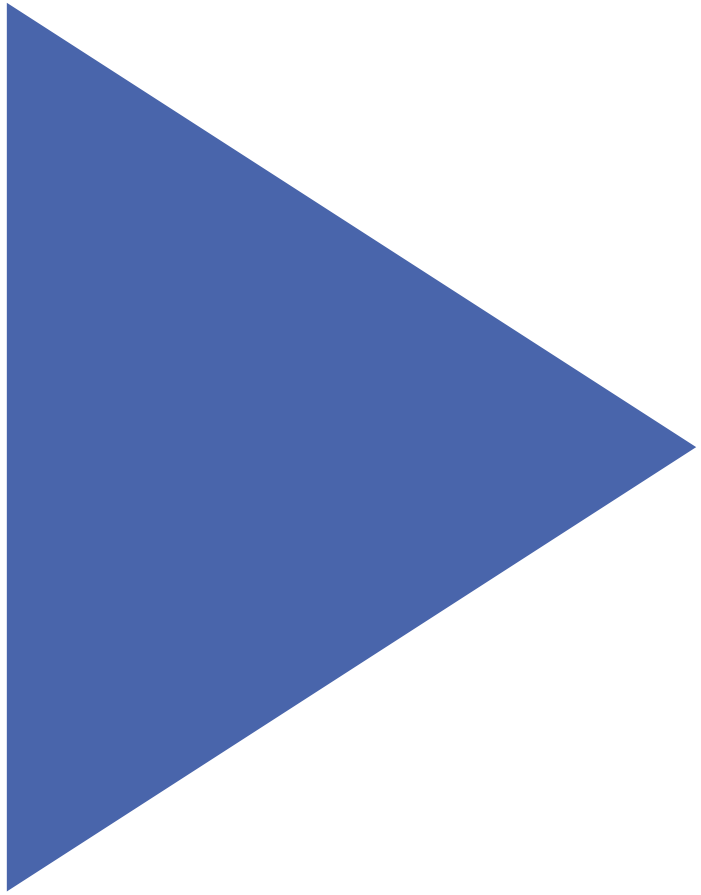
Assessment, conversion and load results for the run

dbo.usp_ArchiveStaleCustomers → PL/pgSQL

```
CREATE OR REPLACE FUNCTION
  public.fn_archive_stale_customers(
    p_inactive_months INTEGER DEFAULT 24,
    OUT p_archived_count INTEGER)
LANGUAGE plpgsql VOLATILE AS $function$
DECLARE v_cutoff TIMESTAMP(3) :=
  now() - p_inactive_months * INTERVAL '1 month';
BEGIN
  UPDATE public.customers SET is_active = FALSE
   WHERE is_active = TRUE AND COALESCE(
     last_purchase_date, created_date) < v_cutoff;
  GET DIAGNOSTICS p_archived_count = ROW_COUNT;
END; $function$;
```

Rules applied: GETDATE() → now(), @@ROWCOUNT → GET DIAGNOSTICS, OUTPUT param → OUT, ISNULL → COALESCE, BIT predicate → BOOLEAN, PascalCase → snake_case

DEMO



THE EASIEST MIGRATION

- Greenfield services
  - Cloud-native architecture
  - Avoid Enterprise core growth
  - AI-first data strategy



WHEN MIGRATION IS DANGEROUS

- Heavy CLR usage
  - Complex SSIS ecosystems
  - Tight latency SLAs
  - Vendor-locked applications
  - Decade(s) of SQL Server investment



FINAL TAKEAWAYS

- Migration is architectural, not syntactic
  - Performance must be rebuilt and takes time
  - Cloud may reintroduce lock-in, so prepare decision makers
  - AI may reduce and ease some burden of migration

CLOSING THOUGHT

Escaping licensing does not eliminate engineering responsibility.

- Success requires discipline, measurement, automation, and governance.

Costs are often shifted from database licensing to cloud and AI.

Having tools that cover both platforms can make the difference.



THANK YOU!

Kellyn Gorman

Multiplatform Engineer and Advocate, Redgate

Kellyn.Gorman@red-gate.com